

BUC3219 Web Server Administration

Phase II Group Report — Final Examination Project

Company Name: MensaHost
Domain: mensahost.tech
Course: BUC3219 Web Server Administration | Semester II AY 2025/2026
Programme: Bachelor of Business Computing — Year 3
Facilitators: Ms. Hajara Namuwaya | Mr. Samuel Ssendi | Mr. Nyero A
Submission Date: 10 May 2026

Group Members

Full Name	Student Number	Role
Wampamba Festo	23/U/18503/EVE	Team Leader and DevOps (Reseller)
Kawere Edrine	23/U/09440/PS	Backup and Recovery Admin
Kyarikunda Bakeine Grace	23/U/10491/PS	UI/UX Designer
Kamariza Hellena	23/U/08844/PS	UI/UX Designer
Talemwa Daniella	23/U/17830/EVE	Security Management
Awori Betsy Hope	23/U/07084/PS	Documentation Lead
Tumusiime Elvin Luke	23/U/18113/PS	Database Management
Mungujakisa Maxwell	23/U/12023/EVE	Database Management
Kirabo Queen Esther	23/U/24679/PS	System Monitoring

1. Company Profile

Company Name: MensaHost
Tagline: Reliable Hosting, Built by Students, Trusted by Practice
Domain: mensahost.tech (registered via Namecheap, managed via Cloudflare DNS)

MensaHost is the group's simulated web hosting company created for this project. The main company website is hosted at `mensahost.tech` and each group member operates their own subdomain as an individual hosting account within the shared environment.

Member Subdomains

Member	Subdomain
Wampamba Festo	festo.mensahost.tech
Kawere Edrine	edrine.mensahost.tech
Kyarikunda Bakeine Grace	grace.mensahost.tech
Kamariza Hellena	hellena.mensahost.tech
Talemwa Daniella	daniella.mensahost.tech
Awori Betsy Hope	betsy.mensahost.tech
Tumusiime Elvin Luke	elvin.mensahost.tech
Mungujakisa Maxwell	maxwell.mensahost.tech
Kirabo Queen Esther	esther.mensahost.tech

2. VPS Details

Parameter	Value
Provider	Contabo Cloud VPS 20 SSD
Data Centre	EU Region
Operating System	AlmaLinux 9.7 (Moss Jungle Cat) x86_64
CPU	AMD EPYC Processor, 6 vCores at 2800 MHz
RAM	12 GB
Storage	200 GB SSD
Public IP Address	185.2.100.162

Parameter	Value
Server Hostname	server.mensahost.tech
CWP Admin Panel	https://185.2.100.162:2031
CWP User Panel	http://185.2.100.162:2083

AlmaLinux 9 was selected over Rocky Linux 9 because CWP's documentation explicitly recommends AlmaLinux as the preferred EL9 target for 2026, and its CloudLinux origins give it better compatibility with CWP's internal scripts and service management.

3. Task 1: CWP Installation and Initial Configuration

What Was Done

The Phase I server was running Rocky Linux 9. Before installing CWP, the VPS was reinstalled via the Contabo customer panel using the AlmaLinux 9 standard image, keeping the same public IP address. A snapshot of the Phase I Rocky Linux state was taken before wiping the disk as a restore point.

After SSHing into the fresh server, the hostname was set using:

```
bash
hostnamectl set-hostname mensahost.tech
```

The entry `185.2.100.162 mensahost.tech mensahost` was added to `/etc/hosts`. The system was updated with `dnf update -y`, and EPEL was installed with `dnf install epel-release -y` along with `wget`, both required by the CWP installer.

The CWP installer was downloaded and executed:

```
bash
cd /usr/local/src
wget -4 http://centos-webpanel.com/cwp-el9-latest
sh cwp-el9-latest
```

The `-4` flag forced IPv4 because the server's IPv6 routing was unreachable. The install compiled Apache and PHP from source and took approximately 25 minutes. After the server rebooted, CWP was confirmed running via `systemctl status cwpsrv`.

Post-installation configuration completed:

- Admin and alert email set to `festotechug@gmail.com`
- Timezone set to `Africa/Kampala` (EAT, UTC+3)
- PHP timezone updated in both `/usr/local/php/php.ini` and `/usr/local/cwp/php71/php.ini`
- CWP update channel set to Stable
- Web server mode set to Nginx and Apache reverse proxy

Error encountered and resolved: After selecting the Nginx and Apache reverse proxy mode and rebuilding, Apache failed to start with the error: `Cannot load modules/mod_rpaf.so: no such file or directory`. The CWP rebuild had written a configuration referencing `mod_rpaf.so`, but the module was never compiled. `mod_rpaf` is an unmaintained third-party module. The fix was to rename the broken config to `rpaf.conf.disabled` and create a replacement config at `/usr/local/apache/conf.d/remoteip.conf` using Apache 2.4's built-in `mod_remoteip`, which was already present in the modules directory. After confirming `Syntax OK` with `apachectl -t`, both Nginx and Apache started correctly.

A second update script error (`service: command not found`) occurred because AlmaLinux 9 does not include the legacy SysV init compatibility layer by default. Installing `dnf install initscripts -y` resolved it.

CWP vs cPanel

CWP is a free control panel built for EL-based Linux distributions. cPanel is a commercial product that costs approximately \$15-45 per month per server depending on the licence tier. Both provide a web interface for managing hosting accounts, but cPanel is more mature, better documented, and used by most commercial web hosts. Five tasks CWP handles that were done manually in Phase I: virtual host creation per account, SSL certificate issuance via AutoSSL, DKIM key generation per domain, DNS zone management, and service restarts after configuration changes.

Screenshots required: CWP installer running in terminal, CWP admin panel login page, CWP dashboard after login, web server mode selection screen, `ss -tlnp` showing Nginx on 80/443 and Apache on 8181.

4. Task 2: User Accounts and Hosting Packages

What Was Done

Default package deleted. CWP shipped with a pre-existing default package that did not match the project's resource requirements and had no accounts assigned. It was deleted before creating the group package.

MensaHost-Student package created under Packages > Add New Package with the following limits:

Resource	Limit	Justification
Disk Quota	5120 MB (5 GB)	Nine accounts at 5 GB each totals 45 GB. The 200 GB VPS leaves over 140 GB after system and CWP usage.
Bandwidth	51200 MB (50 GB/month)	Sufficient for student subdomain sites with low to moderate traffic.
FTP Accounts	2	One for deployment, one spare.
Email Accounts	10	Covers all professional email accounts per member.
MySQL Databases	5	Covers the application database with room for testing.
Subdomains	10	Allows each member to create sub-subdomains if needed.
Addon Domains	0	Members manage only their assigned subdomain.
Inodes	100,000	Prevents any single account from exhausting the server's inode table.

Nine CWP user accounts were created, one per group member, each assigned to the MensaHost-Student package:

Username	Primary Domain
festo	festo.mensahost.tech
edrine	edrine.mensahost.tech
grace	grace.mensahost.tech
hellena	hellena.mensahost.tech
daniella	daniella.mensahost.tech
betsy	betsy.mensahost.tech
elvin	elvin.mensahost.tech
maxwell	maxwell.mensahost.tech
esther	esther.mensahost.tech

Festo's account was also configured as a Reseller, giving him management visibility over other accounts through the user panel at port 2083. The remaining eight members are standard users who can only see and manage their own account data. Both edrine and festo logged into the CWP user panel successfully and each dashboard correctly showed only that user's package limits and resources.

Resource Limits and Account Isolation

Resource limits prevent any one account from consuming resources that other accounts depend on. Without a disk quota, a single account uploading large files could fill the server and bring down every other site. Without an inode limit, a runaway script creating thousands of small files could exhaust the filesystem's inode table even if disk space appeared available. Account isolation means each CWP user can only see their own files, databases, and email — they cannot browse another account's document root or connect to another account's database. This is the foundation of shared hosting security.

Screenshots required: MensaHost-Student package in the packages list, all nine accounts listed under User Accounts, edrine's CWP user panel dashboard, festo's CWP user panel dashboard.

5. Task 3: Web Server Configuration via CWP

What Was Done

Web server mode confirmed as Nginx and Apache reverse proxy via WebServer Settings >

WebServer Manager. In this mode, Nginx listens on ports 80 and 443 as the public-facing server and passes requests to Apache running on port 8181. Apache processes PHP and static files and returns the response to Nginx, which delivers it to the browser. This mode was selected because it combines Nginx's performance and connection-handling efficiency with Apache's full support for `.htaccess` files and `mod_rewrite`, which the PHP application uses for URL rewriting.

All domains added. When each CWP user account was created, CWP automatically generated Apache virtual host and Nginx proxy configurations for that account's primary domain. These are tracked under User Accounts, not the "List Domains" page, which only shows addon domains. The group main domain `mensahost.tech` required two extra steps:

1. The server hostname had to be changed from `mensahost.tech` to `server.mensahost.tech` because CWP blocked the bare domain from being added as a hosted account when it matched the registered hostname. This was fixed with `hostnamectl set-hostname server.mensahost.tech`.
2. A DNS A record for `server.mensahost.tech` was added in Cloudflare pointing to `185.2.100.162`.

After this, `mensahost.tech` was added via Domains > Add Domain, assigned to user `festo` with folder path `/public_html/mensahost.tech`, creating the document root at `/home/festo/public_html/mensahost.tech/` — separate from festo's personal subdomain at `/home/festo/public_html/`.

Document roots and permissions follow the CWP standard path

`/home/username/public_html`. Ownership is `username:username`, directory permissions are `755`, and file permissions are `644`. This allows the web server to read files while preventing other users from writing to any account's files.

All 10 URLs verified loading: `mensahost.tech`, `www.mensahost.tech`, and all nine member subdomains. A Cloudflare 526 error appeared initially because Cloudflare's SSL mode required a valid origin certificate, which did not exist yet. Setting Cloudflare to Flexible mode resolved this for Task 3. It was corrected to Full Strict in Task 5 after certificates were issued.

`.htaccess` redirect tested on festo's subdomain. A `Redirect 301 /about http://festo.mensahost.tech/` directive was placed in `/home/festo/public_html/.htaccess`. An earlier version that included `Options +FollowSymLinks` caused a 500 error because CWP's Apache template does not grant `AllowOverride Options`. Removing that line and keeping only the `Redirect` directive resolved it. Verified with `curl -v` returning `HTTP/1.1 301 Moved Permanently` with the correct `Location` header.

Screenshots required: WebServer Manager showing Nginx and Apache mode, CWP domain list showing all accounts, browser showing all 10 domains loading, curl output showing the 301 redirect.

6. Task 4: GitHub Deployment

What Was Done

A deploy SSH key pair was generated on the server:

```
bash  
ssh-keygen -t ed25519 -C "festo@mensahost.tech" -f /root/.ssh/github_deploy
```

The `ed25519` algorithm produces a compact, modern key. The public key was added to the GitHub repository under Settings > Deploy Keys with read access. An SSH config entry directed all GitHub connections to use that key. Authentication was confirmed with `ssh -T git@github.com`.

Each member's repository was cloned directly into their document root:

```
bash  
git clone git@github.com:mena-host/mensahost-app.git /home/username/public_html/mer
```

Correct ownership was set after each clone:

```
bash  
chown -R username:username /home/username/public_html/mensa  
find /home/username/public_html/mensa -type d -exec chmod 755 {} \  
find /home/username/public_html/mensa -type f -exec chmod 644 {} \;
```

Routing fix applied. The application's `mensa/index.php` reads `$_SERVER['HTTP_HOST']` to detect the subdomain and route to the correct member page. The routing block had to appear before `require_once 'db.php'` and use `__DIR__` for file paths. The group main domain at `mensahost.tech/mensa/` was incorrectly redirecting to festo's subdomain — this was corrected to serve the main portal directly. After the fix, visiting any member's subdomain loads their individual page, and `mensahost.tech/mensa/` loads the full team portal showing all nine members.

Live update demonstrated. A text change was committed and pushed to `origin main`, then pulled on the server with `git pull origin main`. The change appeared in the browser immediately with no server restart needed.

Git-Based Deployment vs FTP

FTP transfers files one at a time with no record of what changed, no version history, and no way to undo an overwrite. If a bad file is uploaded via FTP, recovery requires finding a local backup and re-uploading. Git-based deployment is atomic: every change is a commit with a message, author, and timestamp. A bad deployment is reversed with `git revert`. The full history shows exactly what changed, when, and by whom.

A CI/CD pipeline extends this further by automating the deployment step. A GitHub Actions workflow can be configured to SSH into the server and run `git pull` automatically whenever code is pushed to the main branch. This removes the manual step entirely: a developer pushes code to GitHub and the server updates without anyone logging in. Combined with automated tests that run before the pull, this means only code that passes testing ever reaches production.

Screenshots required: SSH config file, `ssh -T git@github.com` success output, repository cloned in document root, browser showing deployed application, before and after screenshots of the live update.

7. Task 5: DNS and SSL via CWP

What Was Done

DNS review. The CWP DNS Manager at DNS Functions > List DNS Zones showed the zone file for `mensahost.tech`. All required records were already in place from CWP's automatic zone generation:

Record Type	Name	Value
A	@	185.2.100.162
CNAME	www	mensahost.tech
MX	@	mensahost.tech (priority 0)
TXT	@	v=spf1 +a +mx +ip4:185.2.100.162 ~all
TXT	@	v=DMARC1; p=none
TXT	default._domainkey	DKIM public key
A	webmail, cpanel, cwp	185.2.100.162

No records needed to be added at this stage. The zone was complete.

SSL certificates issued. CWP's AutoSSL at WebServer Settings > SSL Certificates > AutoSSL issued Let's Encrypt certificates for all ten hosted accounts in a single operation. All certificates were signed by Let's Encrypt with an expiry date of 09 August 2026 (89 days from issuance). CWP automatically updated each account's Nginx configuration to enforce HTTPS and redirect HTTP traffic.

Cloudflare updated to Full Strict. After certificates were issued, Cloudflare's SSL/TLS encryption mode was changed from Flexible to Full (Strict). In Flexible mode, traffic between Cloudflare and the origin server was unencrypted. Full Strict requires Cloudflare to verify a valid certificate on the origin, enforcing encryption across the entire request path. All 10 domains were verified returning `(HTTP/2 302)` via curl, confirming the HTTP-to-HTTPS redirect was active on every domain.

Manual vs CWP DNS and SSL Management

In Phase I, each DNS record was created manually in the Cloudflare dashboard and each SSL certificate required a separate Certbot command. The Apache virtual host configuration then had to be updated manually to add the certificate paths and the HTTPS redirect block. In Phase II, AutoSSL issued all ten certificates in one operation and CWP inserted the certificate paths into every virtual host automatically. The DNS Manager gives a single view of all zone records without switching between tools.

The main risk that remains is that CWP maintains its own BIND DNS server separately from Cloudflare. Any record added only inside CWP's DNS Manager will not propagate publicly — Cloudflare is the authoritative nameserver, so records must exist there. Certificate renewal is scheduled automatically by CWP, but failures would go unnoticed without active monitoring of the panel or server logs.

Screenshots required: CWP DNS zone editor showing all records, AutoSSL list showing all 10 accounts with certificates and expiry dates, Cloudflare SSL/TLS overview on Full Strict, terminal showing all 10 domains returning HTTP/2 302.

8. Task 6: Mail Server and Professional Email via CWP

What Was Done

Postfix and Dovecot were confirmed running via `(systemctl status postfix dovecot)`. During this check, the logs revealed that `(postfix-mysql)` was missing on AlmaLinux 9, preventing MySQL-based virtual mailbox lookups. Installing `(dnf install postfix-mysql -y)` and restarting Postfix cleared the warnings.

Nine professional email accounts were created through CWP Admin > Email > Email Accounts > Add new domain mail, all at `(mensahost.tech)` under the `(festo)` user account (the only account

linked to the main domain in CWP):

betsy@mensahost.tech, daniella@mensahost.tech, edrine@mensahost.tech,
elvin@mensahost.tech, esther@mensahost.tech, festo@mensahost.tech,
grace@mensahost.tech, hellena@mensahost.tech, maxwell@mensahost.tech

All accounts were confirmed Active in the mailbox list with 512 MB quota each, stored under /var/vmail/mensahost.tech/.

DKIM configured. CWP's DKIM Manager was found at Email > DKIM Manager. The DKIM public key for mensahost.tech was retrieved and the following TXT record was added in Cloudflare DNS with proxy status set to DNS only:

- Name: default._domainkey
- Content: v=DKIM1; k=rsa; p=[public key value]

Propagation was confirmed with dig TXT default._domainkey.mensahost.tech +short. After running Check Records in CWP's DKIM Manager, all indicators for mensahost.tech turned green.

Two delivery errors diagnosed and fixed:

Error 1 — Postfix hostname conflict. myhostname was set to mensahost.tech, which appeared in both mydestination and virtual_mailbox_domains. When a domain is in mydestination, Postfix treats it as local and routes mail to /var/spool/mail/ instead of /var/vmail/. Fixed with:

```
bash

postconf -e "myhostname = server.mensahost.tech"
```

Error 2 — LMTP socket path mismatch. Postfix was looking for the Dovecot LMTP socket at /var/spool/postfix/private/dovecot-lmtp while Dovecot was creating it at /var/run/dovecot/lmtp. Additionally, Dovecot's mail_uid and mail_gid were not set, preventing it from writing to /var/vmail/. Fixed with:

```
bash

postconf -e "virtual_transport = lmtp:unix:/var/run/dovecot/lmtp"
```

mail_uid = 1000 and mail_gid = 12 were added to /etc/dovecot/conf.d/10-mail.conf.

After these fixes, an email from festo@mensahost.tech to grace@mensahost.tech delivered correctly and appeared in Grace's Roundcube inbox. An email from festo@mensahost.tech to an external Gmail address arrived in Gmail's Primary inbox, confirming DKIM and SPF passed

Gmail's authentication checks. A Gmail reply back to `festo@mensahost.tech` also delivered successfully after a TLS certificate mismatch was resolved by pointing Postfix to the matching ECC certificate pair from CWP's acme.sh store at `/root/.acme.sh/cwp_certs/festo.mensahost.tech_ecc/`.

SPF, DKIM, and Why Both Are Needed

SPF (Sender Policy Framework) works by publishing a DNS TXT record that lists the IP addresses authorised to send email for a domain. When a receiving mail server gets a message claiming to be from `mensahost.tech`, it checks whether the sending IP is in the SPF record. If not, the message is likely spoofed. SPF protects the envelope sender — the address used during the SMTP handshake.

DKIM (DomainKeys Identified Mail) works differently. Instead of checking the sending IP, DKIM attaches a cryptographic signature to the email headers using a private key held on the sending server. The receiving server retrieves the corresponding public key from DNS and verifies the signature. This proves the message was not modified in transit and that the sender controls the domain.

Neither is sufficient alone. SPF breaks when email is forwarded, because the forwarding server's IP is not in the original domain's SPF record. DKIM survives forwarding because the signature travels with the message headers. Conversely, SPF catches spoofed messages that do not use DKIM. Together, they give receiving servers two independent ways to verify legitimacy. DMARC ties both together by letting domain owners specify what to do when SPF and DKIM both fail — quarantine the message, reject it, or just report it.

Screenshots required: CWP mailbox list showing all nine accounts Active, DKIM Manager showing mensahost.tech with all green indicators, Cloudflare DNS record for default._domainkey, dig output confirming DKIM propagation, Grace's Roundcube inbox with festo's email opened, festo's Roundcube inbox with the Gmail reply opened, Gmail inbox showing the outbound email in the Primary tab.

9. Task 7: Database and phpMyAdmin via CWP

What Was Done

Nine databases were created through CWP Admin > SQL Services > MySQL Manager > Databases tab using the Add new database button. For each member, the user account was selected from the dropdown and `mensadb` was entered as the database name. CWP applied its `USERNAME_DBNAME` naming convention automatically, producing: `betsy_mensadb`, `daniella_mensadb`, `edrine_mensadb`, `elvin_mensadb`, `esther_mensadb`, `festo_mensadb`, `grace_mensadb`, `hellena_mensadb`, and `maxwell_mensadb`.

CWP's Users tab in MySQL Manager does not expose an Add User form in CWP7 — it only allows password changes on existing users. To avoid accidentally modifying CWP's internal system users, all nine application database users were created via a single heredoc command fed directly to MySQL:

```
sql

CREATE USER 'festo_mensaadmin'@'localhost' IDENTIFIED BY 'mensa@user';
GRANT ALL PRIVILEGES ON festo_mensadb.* TO 'festo_mensaadmin'@'localhost';
-- (repeated for all nine members)
FLUSH PRIVILEGES;
```

Each user is bound to `localhost`, preventing remote connections entirely.

The group schema file contained `CREATE DATABASE`, `CREATE USER`, and `GRANT` statements that would conflict with the databases and users already created. These lines were stripped before importing:

```
bash

grep -vE "(CREATE DATABASE|CREATE USER|GRANT|FLUSH|USE)" /tmp/schema.sql > /tmp/sch
```

The clean schema was then imported into each member's database with a loop. All imports completed silently, confirming success. The `members` table in each database contained all nine group member rows after import.

phpMyAdmin is accessible at `https://185.2.100.162:2031/pma/`. When logged in as `festo_mensaadmin`, the panel shows only `festo_mensadb` and `information_schema`. The same isolation was confirmed for `grace_mensaadmin` and `hellena_mensaadmin`. The `information_schema` appearing for all users is expected MariaDB behaviour since it is a read-only system metadata database automatically granted to every user.

The live application at `https://mensahost.tech/mensa/` rendered all nine member cards dynamically from the database, and individual subdomain pages also loaded correctly.

CLI vs CWP Database Management

In Phase I, every database, user, and grant was created directly in the MySQL shell. This gave full control but required knowing the exact syntax for each operation and running commands carefully to avoid mistakes. There was no visual confirmation of what existed until a `SELECT` query was run.

CWP's MySQL Manager provides a visual list of all databases and their owners, making it easy to see the current state at a glance. Creating a database through the panel automatically enforces the naming convention and links it to the correct account. The trade-off is that the panel hides

complexity: when CWP's Add User form was unavailable in CWP7, the only way to create users was through the CLI anyway. For batch operations across nine accounts, the CLI heredoc approach was faster than clicking through the panel nine times.

Two shared hosting database risks: First, privilege escalation through misconfigured database users. If a user is granted privileges on `(*.*)` instead of only their own database, they can read or modify every other account's data. Second, resource exhaustion from unoptimised queries. In a shared MariaDB instance, one account running a slow full-table scan on a large dataset consumes CPU and memory that degrades response time for all other accounts on the server.

Screenshots required: CWP MySQL Manager showing all nine databases, terminal showing all nine users created, phpMyAdmin logged in as festo_mensadmin showing only festo_mensadb, phpMyAdmin showing the members table with data, live application rendering member data in the browser.

10. Task 8: CSF Firewall and Security Hardening via CWP

What Was Done

CSF version 15.00 was pre-installed by CWP but was non-functional due to a broken iptables path. Running `(csf -v)` returned a fatal error because the path `(/sbin/iptables)` in `(csf.conf)` was incorrect. On AlmaLinux 9, the actual binary lives at `(/usr/sbin/iptables)` because `(/sbin)` is a symlink to `(/usr/sbin)` on RHEL 9 family systems. Two `(sed)` commands corrected the `(IPTABLES)` and `(IPTABLES6)` paths in `(/etc/csf/csf.conf)`. After the fix, `(csf -v)` returned `(csf: v15.00 (CentOS Web Panel))` cleanly.

Port audit. A full port audit was run with `(ss -tulnp)` before touching any firewall rules. Three problems were identified: MariaDB was bound to `(*:3306)` (accessible from the public internet), rpcbind on port 111 had no purpose on a web server, and ports 8181/8443 (Apache backend) were publicly reachable even though Apache is only supposed to receive traffic internally from Nginx.

Three targeted fixes applied before configuring CSF rules:

- MariaDB restricted to localhost by adding `(bind-address = 127.0.0.1)` to `(/etc/my.cnf.d/mariadb-server.cnf)`. Confirmed with `(ss -tulnp | grep 3306)` showing `(127.0.0.1:3306)` instead of `(*:3306)`.
- Passive FTP range enabled in pure-ftpd by uncommenting `(PassivePortRange 30000 50000)` in `(/etc/pure-ftpd/pure-ftpd.conf)`. Without a defined range, CSF cannot selectively allow passive FTP data connections.
- `(TESTING = "0")` confirmed in `(csf.conf)` to ensure rules are enforced rather than just loaded.

CSF port rules configured. The final `(TCP_IN)` value in `(csf.conf)`:

20, 21, 22, 25, 53, 80, 110, 143, 443, 465, 587, 853, 993, 995, 2030, 2031, 2082, 2083, 2086, 2087, 2095

Ports 8181 and 8443 were deliberately excluded (Apache is internal only). Port 111 was excluded (rpcbind serves no purpose). Port 3306 was excluded (MariaDB now binds to localhost only). The passive FTP range 30000:50000 was added. `UDP_IN` was set to `20, 21, 53, 80, 443, 853`.

LFD configured for SSH brute-force detection:

- `LF_SSHD = "3"` — three failed SSH attempts triggers a block
- `LF_INTERVAL = "60"` — failures must occur within 60 seconds
- `LF_SSHD_PERM = "3600"` — block lasts one hour
- `LF_EMAIL_ALERT = "1"` — alert sent to `festotechug@gmail.com` on each block

Admin IPs were whitelisted in both `csf.allow` (firewall layer ACCEPT rules) and `csf.ignore` (LFD layer protection). The distinction matters because with `IGNORE_ALLOW = "0"`, LFD does not automatically respect `csf.allow` entries. Only IPs in `csf.ignore` are guaranteed never to be blocked by LFD regardless of failure count.

LFD block test. SSH was attempted four times from the admin machine using the username `hacker` with wrong passwords. The `lfd.log` at `/var/log/lfd.log` logged each attempt as `Failed SSH login from [IP] - ignored`, confirming LFD detected all failures and respected the `csf.ignore` entry. The same log revealed the server was under active brute-force attack from real external IPs throughout the test session. Between 19:43 and 20:05 on 11 May 2026, LFD blocked IPs from Lithuania, the Netherlands, and Andorra under `[LF_SSHD]` after 3 failures in 60 seconds. The `/var/log/secure` log confirmed these were genuine SSH authentication failures using credential-stuffing usernames including `root`, `oracle`, and `grid`.

Security audit via Lynis. CWP's built-in Lynis and RKHunter modules require the CWPpro paid licence, so both tools were installed via the CLI. The initial Lynis run scored the server at a hardening index of 66 out of 100 across 268 tests, with 2 warnings.

Two recommendations addressed:

Fix 1 — BIND version disclosure (NAME-4210). Running `dig @185.2.100.162 version.bind chaos txt` returned the exact BIND version string, exposing it to anyone who queried it. Adding `version "none";` inside the `options {}` block of `/etc/named.conf` and restarting `named` caused the same query to return `"none"` instead. The version is still answered (so the query does not time out suspiciously) but contains no exploitable information.

Fix 2 — SMTP banner disclosure (MAIL-8818). The default Postfix banner included the mail software name in every SMTP greeting. Running `postconf -e 'smtpd_banner = $myhostname`

`ESMTP'` removed the `{mail_name}` variable from the banner. After restarting Postfix, the banner no longer reveals which mail software is running.

The second Lynis run returned a hardening index of 67, with the Warnings section empty and the Malware scanner component showing active because installing RKHunter satisfied that check.

CSF vs UFW

In Phase I, UFW managed firewall rules and Fail2ban handled brute-force detection as two completely separate tools with separate config files, separate restart commands, and separate whitelisting logic. CSF combines both functions in a single tool. The firewall rules and LFD failure thresholds are both in `/etc/csf/csf.conf`. Reloading rules requires one command (`csf -r`) instead of two. CSF also integrates with the CWP web interface, allowing an admin to view active rules and blocked IPs without using the terminal.

The downside is that CSF is more complex to diagnose when it breaks. The initial iptables path error required understanding AlmaLinux 9's filesystem symlink structure. UFW on Ubuntu handles this transparently. For this server environment — AlmaLinux 9 with CWP — CSF is the correct choice. For a simpler Ubuntu setup without a control panel, UFW plus Fail2ban remains the better option.

Screenshots required: `csf -v` showing version, `ss -tlnp | grep 3306` showing localhost-only binding, CSF rules active in terminal, LFD config in `csf.conf`, `lfd.log` showing real attacks being blocked, `dig version.bind` before and after, Lynis scores before and after.

11. Task 9: Performance Monitoring and Log Analysis via CWP

What Was Done

CWP dashboard monitoring. The built-in CWP dashboard at `https://185.2.100.162:2031` displayed real-time server metrics:

- CPU usage: 0% at idle
- RAM with cache: 0.76 GB / 12 GB (6.3%)
- RAM without cache: 0.50 GB / 12 GB (4.2%)
- Swap: 0 GB / 0 GB (no swap pressure)
- Disk IO: 0.00 at idle
- Active processes: 199
- Postfix mail queue: 0 (no backlog)

Services status. All seven core services confirmed Active in the CWP Services Status panel: Apache Webserver, Nginx Reverse Proxy, FTP Server, MySQL Database Server, BIND DNS Server, SSH Server, and Crontab.

Apache service issue diagnosed and resolved. During this task, Apache showed as Failed in the dashboard. The error log at `/usr/local/apache/logs/error_log` revealed a certificate and private key modulus mismatch. The fix was to regenerate a matching self-signed certificate pair with `openssl req -x509`, then create a proper systemd unit file for CWP's Apache binary at `/usr/local/apache` since CWP does not use the system `httpd` package. After enabling and starting the service via systemd, Apache returned Active status in both the terminal and CWP dashboard.

Disk details:

Filesystem	Mount Point	Usage	Size
/dev/sda4	/	Low	199 GB
/dev/sda3	/boot	30%	960 MB
/dev/sda2	/boot/efi	Low	200 MB

Process check. High-resource processes were checked with `ps aux --sort=-%cpu` and `ps aux --sort=-%mem`. No unexpected processes were found. MariaDB was the highest memory consumer at 23880 RES, which is normal for a database server.

Listening port audit. `ss -tlnp` confirmed all expected services with no unknown processes. Active ports: 21 (FTP), 22 (SSH), 25/465/587 (Postfix), 53 (BIND), 80/443 (Nginx), 110/143/993/995 (Dovecot), 2030/2031 (CWP admin), 3306 (MariaDB, localhost only), 8181/8443 (CWP Apache, internal).

Log file locations:

Log	Path
Apache access log	<code>/usr/local/apache/logs/access_log</code>
Apache error log	<code>/usr/local/apache/logs/error_log</code>
Nginx access log	<code>/var/log/nginx/access.log</code>
Nginx error log	<code>/var/log/nginx/error.log</code>
CWP service log	<code>/usr/local/cwpsrv/logs/error_log</code>

Security hardening applied: `hidepid=2` was mounted on `/proc` via `/etc/fstab` to prevent non-root users from seeing processes owned by other users. `mod_security` and `mod_security_crs` were installed via `dnf` to provide WAF capability at the Apache layer.

Log Analysis Interpretation

The access log showed consistent low-volume traffic from the group members' IPs during testing, with clean 200 responses for all application paths. No 500 errors appeared in the application logs, confirming the routing fix in Task 4 was stable. The `lfd.log` from Task 8 showed the most significant activity: repeated SSH authentication attempts from automated scanners operating on consistent two-minute intervals, indicating distributed botnet behaviour targeting port 22 with credential-stuffing attacks. LFD blocked each source IP within 60 seconds of the third failure, with no manual intervention required. The Postfix mail queue remaining at 0 throughout monitoring confirmed no delivery backlog or mail loop.

CWP Log Viewer vs Command-Line Analysis

CWP's log viewer provides per-account log access through a web interface, which is useful for giving a hosting client access to their own site logs without SSH. It also integrates with the services status panel so an admin can click through to error logs directly from a failed service indicator. What it cannot do is run custom queries across multiple logs simultaneously.

Command-line tools like `grep`, `awk`, and `tail` can process millions of log lines in seconds, combine patterns across multiple files, and produce aggregated statistics like top IPs, most requested URLs, or error frequency counts. For a hosting administrator, both approaches are complementary: the CWP viewer handles day-to-day browsing and account-level access, while CLI tools are necessary for any serious log analysis or incident investigation.

Screenshots required: CWP dashboard showing all metrics, all seven services Active, `ps aux` output, `ss -tlnp` output, `df -hT` output, `htop` running, Apache Active status after fix.

12. Reflection

The most difficult part of Phase II was the interaction between CWP's internal configuration and the standard Linux service management tools. CWP maintains its own Apache binary at `/usr/local/apache` and its own PHP build, separate from the system packages. When something broke — like the Apache certificate mismatch in Task 9 or the CSF iptables path in Task 8 — the fix required knowing which binary, which config file, and which service unit was actually in use, rather than assuming the standard system paths applied. This was less of a problem in Phase I, where every component was installed directly and its location was known from the installation step.

What the group would do differently: configure the systemd unit for CWP's Apache binary from the start of Phase II, not as a late fix during Task 9. CWP does not register its Apache with systemd by default, which means `systemctl status httpd` gives no information about whether CWP's web server is running. Having a proper unit from the beginning would have made all service status checks consistent.

The main takeaway is that control panels do not remove the need to understand the underlying system. They automate the routine tasks — account creation, SSL issuance, virtual host generation — but when something goes wrong, the administrator still needs to read the error log, trace the issue to a specific binary or config file, and fix it at the system level. Phase I built exactly the foundation needed to do that.

13. Live Links

Site	URL
Group main site	https://mensahost.tech
Group application	https://mensahost.tech/mensa/
Festo	https://festo.mensahost.tech
Edrine	https://edrine.mensahost.tech
Grace	https://grace.mensahost.tech
Hellena	https://hellena.mensahost.tech
Daniella	https://daniella.mensahost.tech
Betsy	https://betsy.mensahost.tech
Elvin	https://elvin.mensahost.tech
Maxwell	https://maxwell.mensahost.tech
Esther	https://esther.mensahost.tech

All sites serve over HTTPS with valid Let's Encrypt certificates and load the MensaHost team member portal dynamically from the database.

14. References

- CWP Documentation: <https://docs.control-web-panel.com>
- AlmaLinux 9 Release Notes: <https://wiki.almalinux.org/release-notes/9.7.html>
- CSF Documentation: <https://download.configserver.com/csf/install.txt>
- Postfix Configuration Parameters: <https://www.postfix.org/postconf.5.html>
- Dovecot Configuration Guide: <https://doc.dovecot.org>
- Let's Encrypt Documentation: <https://letsencrypt.org/docs/>
- Apache mod_remoteip: https://httpd.apache.org/docs/2.4/mod/mod_remoteip.html
- Lynis Documentation: <https://cisofy.com/documentation/lynis/>
- Git Documentation: <https://git-scm.com/doc>
- MariaDB Server Documentation: <https://mariadb.com/kb/en/configuring-mariadb-for-remote-client-access/>